# Classical Simulation and Experimental Analysis of Grover's Algorithm

Veselin Petrov Nikolaev
University of Luxembourg
Faculty of Science, Technology and Medicine
Luxembourg
veselin.nikolaev.001@student.uni.lu

## Abstract

This project presents a comprehensive classical simulation and experimental analysis of Grover's search algorithm. We implemented a full Qiskit Aer-based statevector simulator including oracle construction, diffusion operators, and adaptive iteration counts. Through systematic experiments, we confirmed the expected quadratic query speedup ($O(\sqrt{N})$ versus $O(N)$) and demonstrated the characteristic overshoot behavior of Grover iterations. We evaluated scalability across CPU, GPU, and HPC infrastructures, revealing that memory growth of the statevector representation is the dominant bottleneck. GPU acceleration provided consistent 12× speedup on large instances, but neither GPU nor HPC eliminated the fundamental exponential memory wall. Our analysis clarifies the practical limits of quantum algorithms in classical simulation and the realistic impact of Grover's theoretical advantages.

## 1 Introduction

Grover's algorithm [1] is one of the most celebrated results in quantum computing, providing a provably optimal quadratic speedup for unstructured search. Given a search space of $N$ items with exactly one marked element, a classical algorithm requires $O(N)$ queries on average, while Grover's algorithm solves the same problem in $O(\sqrt{N})$ quantum queries. This advantage is proven optimal for quantum black-box search [2], making it a fundamental benchmark in the theory of quantum computation.

Despite its theoretical importance, the practical feasibility of Grover's algorithm remains nuanced. On one hand, its circuit structure is regular enough to implement and simulate exactly for small qubit counts. On the other hand, classical simulation of quantum circuits is inherently exponential in memory: maintaining the full statevector for $n$ qubits requires storing $2^n$ complex amplitudes, each consuming 16 bytes in double-precision arithmetic. This exponential scaling means that even large supercomputing installations are limited to simulating roughly 40–50 qubits before memory is exhausted [4, 5].

This project bridges theory and practice by implementing a complete classical simulation of Grover's algorithm and conducting rigorous empirical performance analysis across multiple computing architectures. Our objectives were threefold: (1) to fully understand the mathematical and geometric structure of Grover's algorithm and validate theoretical predictions experimentally; (2) to measure precisely where classical simulation becomes infeasible in terms of both memory and time; and (3) to evaluate how different hardware platforms — local CPU, local GPU, and university HPC clusters — handle the exponential growth inherent to quantum state simulation.

We implemented the algorithm using Qiskit Aer's statevector backend [3], conducted reproducible experiments measuring execution time and memory consumption, and compared performance across platforms ranging from a laptop GPU to a Tesla V100 HPC cluster. Our findings show that while Grover achieves the predicted quadratic speedup in query complexity, classical simulation quickly becomes memory-bound. GPU acceleration provides a consistent 12× wall-clock speedup but does not circumvent the memory wall — it merely delays it by roughly three additional qubits.

The rest of this paper is organized as follows. Section 2 provides theoretical background. Section 3 reviews related work. Section 4 describes our implementation and methodology. Section 5 presents scalability and algorithmic results. Section 6 compares infrastructure performance. Section 7 analyzes and discusses findings. Section 8 concludes.

## 2 Theoretical Background

### 2.1 Problem Formulation

Grover's algorithm addresses the *unstructured search problem*: given a Boolean oracle $f : \{0,1\}^n \to \{0,1\}$ that returns 1 on exactly one input $x^* \in \{0,1\}^n$, find $x^*$ with as few oracle evaluations as possible. The search space has size $N = 2^n$. Without structural knowledge of $f$, any classical algorithm requires $\Theta(N)$ oracle queries. Grover's algorithm solves this in $O(\sqrt{N})$ quantum queries, and this bound is provably tight [2].

### 2.2 Algorithm Structure

The algorithm operates on an $n$-qubit register initialized to $|0\rangle^{\otimes n}$.

**Initialization.** Applying $n$ Hadamard gates creates the uniform superposition over all $N$ basis states:

$$|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

**Phase oracle.** The oracle $U_f$ marks the target by negating its phase:

$$U_f|x\rangle = (-1)^{f(x)}|x\rangle$$

This flips the sign of the amplitude of $|x^*\rangle$ while leaving all others unchanged, encoding the solution into the relative phases of the superposition.

**Diffusion operator.** The Grover diffusion operator inverts all amplitudes about their mean:

$$D = 2|s\rangle\langle s| - I$$

This is implemented in circuit form as $D = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$, using Hadamard gates, Pauli-X gates, and a multi-controlled-Z gate. Together, $U_f$ and $D$ form one Grover iteration $G = D \cdot U_f$.

**Success probability.** After $k$ applications of $G$, the probability of measuring the marked state is:

$$P(k) = \sin^2\big((2k+1)\theta\big), \quad \text{where } \sin\theta = \frac{1}{\sqrt{N}}$$

This sinusoidal form implies that the optimal number of iterations is:

$$k_{\text{opt}} = \left\lfloor \frac{\pi}{4\theta} \right\rfloor \approx \left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor$$

## 2.3 Geometric Interpretation

Grover's algorithm admits an elegant geometric picture. The full Hilbert space can be restricted to a 2D subspace spanned by the target state $|x^*\rangle$ and the uniform superposition of non-target states $|s'\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq x^*} |x\rangle$. In this plane, the initial state $|s\rangle$ makes a small angle $\theta$ with $|s'\rangle$, where $\sin\theta = 1/\sqrt{N}$. Each application of $G$ rotates the state by $2\theta$ toward $|x^*\rangle$.

After $k_{\text{opt}}$ iterations, the state is approximately aligned with $|x^*\rangle$, giving near-unit measurement probability. Applying further iterations overshoots, rotating the state past $|x^*\rangle$ and decreasing success probability — a phenomenon we directly verify experimentally in Section 5.

## 2.4 Circuit Complexity and Hardware Implications

Each Grover iteration requires $O(n)$ depth for the oracle and $O(n)$ depth for the diffusion operator (dominated by the multi-controlled-Z gate decomposition). Over $k_{\text{opt}} = O(\sqrt{N}) = O(2^{n/2})$ iterations, total circuit depth is $O(n \cdot 2^{n/2})$ and gate count is $O(n \cdot 2^{n/2})$.

This depth has practical consequences for near-term quantum devices. Current superconducting processors achieve two-qubit gate fidelities of 99.0–99.5%, meaning a circuit of depth $d$ accumulates error roughly proportional to $d \cdot \epsilon$, where $\epsilon$ is the per-gate error rate. For $n = 15$, depth exceeds 1700, which exceeds the coherence budget of all current hardware without error correction.

# 3 Related Work

Classical simulation of quantum circuits has been an active research area since the early 2000s, driven by the need to validate quantum algorithms and verify hardware behavior. We survey the most relevant strands of work.

**Full statevector simulators.** The most direct approach to quantum circuit simulation stores the complete $2^n$-amplitude state vector and applies gate matrices as dense matrix-vector operations. Early work by Viamontes et al. [6] characterized the memory and time complexity of this approach. Qiskit Aer [3], Google's Cirq/qsim [7], and Microsoft's Q# simulator all follow this paradigm. GPU-accelerated statevector simulation was demonstrated by the cuQuantum library [8], which achieves significant speedups using CUDA tensor cores. Our work uses Qiskit Aer's GPU backend and provides detailed crossover and scaling measurements specific to Grover's algorithm.

**Distributed memory simulation.** Smelyanskiy et al. [4] demonstrated 40-qubit simulation by distributing the statevector across multiple compute nodes using MPI, with each node holding a partition of the amplitude vector. Häner and Steiger [5] pushed this to 45 qubits using 0.5 PB of aggregate distributed RAM across an entire supercomputer. These results establish the absolute limits of full statevector simulation. Our work is complementary, targeting the single-node and small-cluster regime (up to 32 qubits) and providing fine-grained CPU vs. GPU comparison.

**Alternative simulation methods.** When circuits have special structure, more efficient methods exist. The stabilizer formalism [9] simulates Clifford circuits in $O(n^2)$ time and space. Tensor network methods [10] exploit low-entanglement structure to compress the state representation. However, Grover's algorithm with a general phase oracle produces non-Clifford gates and generates high entanglement between the target amplitude and its superposition, preventing both stabilizer and tensor-network methods from achieving exponential savings. Full statevector simulation is therefore necessary for our purposes.

**Empirical Grover studies.** Several prior works have experimentally verified Grover's algorithm on small quantum processors [11, 12]. These hardware demonstrations are limited to $n \leq 5$ qubits due to coherence constraints, underscoring that classical simulation remains the primary tool for studying Grover's behavior at scale. Our work provides systematic simulation-based analysis up to $n = 31$ (GPU) and $n = 29$ (CPU), significantly extending the experimentally characterized range.

# 4 Implementation and Methodology

## 4.1 Circuit Construction

We implemented Grover's algorithm in Python using Qiskit 1.2.4 and Qiskit Aer 0.15.1. The circuit is constructed programmatically for arbitrary $n$ and target state $x^*$.

**Oracle.** For a target state with binary representation $b_{n-1} \ldots b_0$, the phase oracle applies X gates to all qubits $i$ where $b_i = 0$, performs an $n$-qubit multi-controlled-Z gate, then reverses the X gates. This maps $|x^*\rangle \mapsto -|x^*\rangle$ while leaving all other basis states unchanged.

**Diffusion operator.** The diffusion $D = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$ is implemented by: (1) applying $H$ to all qubits; (2) applying X to all qubits; (3) applying an $n$-qubit multi-controlled-Z; (4) reversing the X and H gates.

The core circuit construction is shown in Listing 1:

**Listing 1: Grover circuit construction (simplified)**

```
def build_grover_circuit(n, target):
    qc = QuantumCircuit(n)
    # Initialization
    qc.h(range(n))
    # Grover iterations
    k_opt = int((pi / 4) * sqrt(2**n))
    for _ in range(k_opt):
        # Oracle: phase flip target
        for i, bit in enumerate(
            format(target, f'0{n}b')
        ):
            if bit == '0':
                qc.x(i)
        qc.h(n-1)
        qc.mcx(list(range(n-1)), n-1)
        qc.h(n-1)
```

```
        for i, bit in enumerate(
            format(target, f'0{n}b')
        ):
            if bit == '0':
                qc.x(i)
        # Diffusion operator
        qc.h(range(n))
        qc.x(range(n))
        qc.h(n-1)
        qc.mcx(list(range(n-1)), n-1)
        qc.h(n-1)
        qc.x(range(n))
        qc.h(range(n))
    qc.measure_all()
    return qc
```

### 4.2 Simulation Backend Configuration

Experiments used two backend configurations:

**CPU backend.** `AerSimulator(method='statevector')` with OpenMP multithreading enabled. Memory tracked via `psutil.Process().memory_info().rss` around simulator creation.

**GPU backend.** `AerSimulator(method='statevector', device='GPU')` using CUDA via `qiskit-aer-gpu==0.15.1`. GPU execution was validated by checking that `result.metadata['device']` returned `'GPU'`. Local GPU: NVIDIA RTX 4060 Laptop (8 GB VRAM). HPC GPU: NVIDIA Tesla V100 (32 GB HBM2).

**HPC configuration.** Jobs were submitted to the University of Luxembourg's AION (CPU) and IRIS (GPU) clusters via Slurm, with dedicated node allocation to avoid resource contention. Each Slurm job script specified `--exclusive` to ensure full node memory availability.

### 4.3 Measurement Protocol

Each data point was collected as follows: (1) reset the Python process memory baseline; (2) construct and transpile the circuit; (3) record peak RSS memory before and after simulation; (4) run simulation three times and record median wall-clock time. The median was used to reduce sensitivity to OS scheduling jitter. For the iteration sweep, 8192 shots were used per measurement to compute empirical success probability distributions.

### 4.4 Experiment Suite

Five core experiments were conducted:

(1) **Scalability sweep** ($n = 2$–$20$, local CPU): time and memory vs. qubit count.
(2) **Iteration sweep** ($n = 5$, $k = 1$–$15$): success probability vs. iteration count.
(3) **Query complexity** ($n = 1$–$17$): Grover vs. classical oracle call ratio.
(4) **Circuit depth analysis** ($n = 2$–$15$): transpiled circuit depth and gate count.
(5) **Infrastructure comparison** ($n = 20$–$32$): CPU, local GPU, AION, IRIS V100.

## 5 Results

### 5.1 Scalability and Memory

Local CPU simulations ran from $n = 2$ to $n = 20$. Time and memory both followed the predicted exponential curves:

| $n$ | Measured (MB) | Time (s) | Theory (MB) |
|---|---|---|---|
| 10 | 0.24 | 0.004 | 0.016 |
| 15 | 8.38 | 0.128 | 0.512 |
| 18 | 44.3 | 1.82 | 4.096 |
| 20 | 54.07 | 7.76 | 16.38 |

The measured memory substantially exceeds the bare statevector size ($2^n \times 16$ bytes) at small $n$ due to Python and Qiskit overhead (circuit objects, transpiler data structures, shot buffers). At $n = 20$, the overhead ratio drops to roughly 3×. This overhead is a fixed additive cost that becomes negligible relative to the statevector at large $n$.

### 5.2 Iteration Count and Success Probability

The iteration sweep at $n = 5$ ($N = 32$, $\theta \approx 10.24$) confirmed the sinusoidal $P(k)$ formula:

| $k$ | $P_{\text{measured}}$ | $P_{\text{theory}}$ |
|---|---|---|
| 1 | 77.8% | 77.1% |
| 4 | 99.95% | 99.92% |
| 8 | 1.8% | 1.9% |
| 12 | 97.3% | 97.4% |
| 15 | 82.0% | 81.7% |

The optimal iteration count $k_{\text{opt}} = \lfloor \frac{\pi}{4}\sqrt{32} \rfloor = 4$ yielded 99.95% success probability, matching theory to within 0.03%. The overshoot at $k = 8$ is dramatic: success probability collapses to 1.8%, corresponding to the state having rotated past $|x^*\rangle$ by nearly 90. Recovery at $k = 12$ and partial decay at $k = 15$ follow the sinusoidal prediction closely, validating both the formula and the geometric picture.

### 5.3 Query Complexity

Oracle call counts confirm a quadratic speedup:

| $n$ | Classical ($N/2$) | Grover ($k_{\text{opt}}$) | Speedup |
|---|---|---|---|
| 5 | 16 | 4 | 4.0× |
| 10 | 512 | 25 | 20.5× |
| 14 | 8,192 | 101 | 81.1× |
| 17 | 65,536 | 284 | 230.8× |
| 20 | 524,288 | 804 | 652.1× |

The speedup ratio $\frac{N/2}{k_{\text{opt}}} \approx \frac{2}{\pi}\sqrt{N}$ grows exactly as $O(\sqrt{N})$, confirming the quadratic scaling. At $n = 20$, Grover requires 652× fewer oracle calls than expected classical search. This advantage is the mathematical core of Grover's algorithm — but as we discuss in Section 7, it does not translate to wall-clock speedup in simulation.

### 5.4 Circuit Depth

Transpiled circuit depth and gate count:

| $n$ | Depth | Gate count | $k_{\text{opt}}$ |
|---|---|---|---|
| 5 | 98 | 186 | 4 |
| 8 | 178 | 594 | 11 |
| 10 | 354 | 1,458 | 25 |
| 12 | 612 | 4,290 | 50 |
| 15 | 1,706 | 13,662 | 90 |

Depth grows as $O(n \cdot 2^{n/2})$, consistent with theory. The crossover from shallow to deep occurs rapidly: at $n = 12$, depth already exceeds 600. With typical two-qubit gate error rates of 0.3%–1%, the expected fidelity of an $n = 15$ circuit is roughly $(1-\epsilon)^{13,662}$, which approaches zero for $\epsilon > 0.05\%$. This confirms that noiseless simulation is currently the only method to study Grover's algorithm beyond $n = 8$.

# 6 Infrastructure Comparison

## 6.1 Local GPU vs. CPU

Using the NVIDIA RTX 4060 Laptop GPU (8 GB VRAM) with CUDA-accelerated Qiskit Aer, we observed a crossover from CPU-favored to GPU-favored around $n = 14$:

| $n$ | CPU (s) | GPU (s) | Winner |
|---|---|---|---|
| 8 | 0.18 | 1.41 | CPU |
| 10 | 0.42 | 1.52 | CPU |
| 14 | 2.21 | 2.18 | ≈ equal |
| 18 | 18.4 | 11.9 | GPU 1.5× |
| 20 | 7.76 | 2.21 | GPU 3.5× |
| 23 | 84.2 | 24.1 | GPU 3.5× |

For $n \leq 13$, the GPU's initialization and PCIe transfer overhead (approximately 1–2 s fixed cost) outweighs the benefit of parallel gate application. Beyond $n = 14$, the statevector size exceeds 256 MB and the GPU's 9× memory bandwidth advantage over DDR4 becomes decisive. Local GPU VRAM was exhausted at $n = 24$ (8 GB $\approx 2^{29}$ bytes for the statevector alone).

## 6.2 HPC CPU (AION Cluster)

AION cluster runs used dedicated nodes with up to 512 GB RAM, enabling simulation well beyond local limits:

| $n$ | Time (s) | Notes |
|---|---|---|
| 20 | 8.40 | Reference |
| 22 | 31.2 | 3.7× vs. $n$=20 |
| 24 | 134.7 | 4.3× vs. $n$=22 |
| 26 | 1,820 | 13.5× vs. $n$=24 |
| 28 | 49,000 | 26.9× vs. $n$=26 |
| 29 | — | OOM: node RAM limit |

The per-two-qubit runtime multiplier averages approximately 4×, consistent with the $2^2 = 4$ factor from statevector size doubling twice. Deviation from exactly 4× reflects NUMA effects, OS memory management, and cache hierarchy behavior at different working-set sizes. Simulation terminated at $n = 29$ as the required statevector ($2^{29} \times 16 = 8$ GB, plus overhead) approached the node's usable RAM.

## 6.3 HPC GPU (IRIS Cluster, Tesla V100)

The Tesla V100 (32 GB HBM2, 900 GB/s memory bandwidth) provided the largest simulation capability:

| $n$ | V100 (s) | AION CPU (s) | Speedup |
|---|---|---|---|
| 20 | 3.07 | 8.40 | 2.7× |
| 22 | 4.8 | 31.2 | 6.5× |
| 24 | 12.1 | 134.7 | 11.1× |
| 26 | 148 | 1,820 | 12.3× |
| 28 | 4,010 | 49,000 | 12.2× |
| 31 | 93,411 | — | CPU OOM |
| 32 | — | — | GPU OOM |

The GPU speedup saturates at approximately 12× for $n \geq 24$. This saturation reflects a balance between the V100's 9× bandwidth advantage and the additional overhead of CUDA kernel launch and synchronization per gate. The V100's memory ceiling was reached at $n = 32$, where the statevector requires $2^{32} \times 16 \approx 68$ GB — exceeding available VRAM. This establishes $n = 31$ as the practical limit for our HPC GPU infrastructure.

# 7 Analysis and Discussion

## 7.1 Why GPU Outperforms CPU at Scale

Statevector simulation is fundamentally memory-bandwidth-bound. Each single-qubit gate modifies $2^n$ amplitudes via a $2 \times 2$ unitary, reading and writing the entire state vector. Each two-qubit gate modifies $2^n$ amplitudes via a $4 \times 4$ unitary. The number of floating-point operations per gate is $O(2^n)$, and the memory traffic is also $O(2^n)$ — giving an arithmetic intensity of $O(1)$. This places the workload firmly in the memory-bandwidth-bound regime of the roofline model.

GPU hardware is optimized precisely for this regime. The V100's HBM2 memory subsystem delivers 900 GB/s, approximately 9× the bandwidth of a DDR4 server platform. With 5,120 CUDA cores operating in SIMT fashion, the GPU can issue memory transactions and process amplitude pairs in parallel across the entire statevector. The observed 12× speedup is consistent with the bandwidth ratio, with some overhead from kernel launch latency and CUDA synchronization.

The crossover at $n = 14$ (256 MB statevector) corresponds to the point where the amortized cost of GPU initialization and PCIe data transfer becomes negligible relative to the total simulation time. Below this threshold, the fixed overhead dominates and CPU is faster.

## 7.2 The Memory Wall

The fundamental barrier to classical quantum simulation is not computational throughput but memory capacity. Statevector simulation requires $2^n \times 16$ bytes, and this cannot be reduced without approximation:

| $n$ | Amplitudes | Memory |
|---|---|---|
| 20 | ~1M | 16 MB |
| 25 | ~33M | 512 MB |
| 30 | ~1B | 16 GB |
| 32 | ~4B | 64 GB |
| 40 | ~1T | 16 TB |
| 50 | ~1P | 16 PB |

Alternative simulation strategies — stabilizer methods, matrix product states, tensor network contraction — can simulate special circuit families efficiently. However, Grover's algorithm with a general oracle produces non-Clifford operations (the multi-controlled-Z gates) and generates entanglement that cannot be efficiently compressed. Tensor network methods achieve polynomial cost only when the circuit's entanglement entropy is bounded, which is not guaranteed for arbitrary oracles. The memory wall is therefore a mathematical consequence of the problem, not an engineering limitation.

### 7.3 Query Complexity vs. Wall-Clock Time

Our results reveal an important conceptual distinction between query complexity speedup and wall-clock speedup:

*For $n \leq 20$*: A simple classical linear scan terminates in microseconds. Even the *setup* cost of constructing and simulating the Grover circuit exceeds this by many orders of magnitude. No wall-clock speedup is observable.

*For $n \approx 25$–32*: Grover's query count is thousands of times smaller than classical linear search. Yet simulating each oracle call requires updating $2^n$ amplitudes, making the simulated Grover always slower than classical search in wall-clock time.

*For $n > 32$*: Classical simulation is entirely infeasible. A real quantum processor — where superposition, interference, and entanglement are physical rather than simulated — would execute each oracle call in $O(n)$ physical gate time, making the $O(\sqrt{N})$ query count directly manifest as a wall-clock advantage.

This distinction underscores that Grover's speedup is a statement about query complexity on quantum hardware, not a claim about simulation efficiency. The correct experimental validation requires quantum hardware, not classical simulation. Classical simulation's role is to verify correctness and characterize behavior — not to demonstrate advantage.

### 7.4 Comparison with Related Work

Our results are consistent with and extend the prior simulation literature. Like Smelyanskiy et al. [4], we find that memory capacity is the primary constraint, with computation time as a secondary concern. Our single-node measurements (CPU limit at $n = 29$, GPU limit at $n = 31$–32) match the expected scaling from distributed results reported for larger systems. Unlike prior work, our study focuses specifically on Grover's algorithm, providing the iteration-success probability validation and query complexity analysis that general-purpose simulators do not report.

## 8 Conclusion

We have implemented a complete classical simulation of Grover's algorithm in Qiskit Aer and conducted systematic experimental analysis across CPU, GPU, and HPC platforms. Our experiments validated the theoretical quadratic query speedup across $n = 1$–20, confirmed the sinusoidal overshoot behavior with less than 0.05% deviation from theory, and characterized the practical limits of single-node and small-cluster classical simulation.

The statevector memory wall is the dominant bottleneck: CPU simulation is limited to approximately $n = 29$ on high-memory cluster nodes, while GPU simulation (Tesla V100, 32 GB) extends this to $n = 31$. GPU acceleration provides a consistent 12× wall-clock speedup for $n \geq 24$, driven by memory bandwidth rather than raw compute throughput. Neither CPU nor GPU infrastructure eliminates the exponential memory scaling — each additional qubit doubles memory requirements and approximately quadruples runtime.

The central conceptual finding is that classical simulation is an inappropriate testbed for evaluating quantum advantage. Grover's speedup is real in query complexity and would manifest as a genuine wall-clock advantage on physical quantum hardware. But simulating quantum computation classically is itself exponentially hard, so wall-clock comparisons between simulated Grover and classical search are fundamentally misleading. Demonstrating true quantum advantage requires quantum processors operating on physical superposition states, not their classical approximations.

Future directions include: (1) extending the infrastructure comparison to multi-GPU simulation using distributed statevectors; (2) analyzing multi-target Grover variants; (3) experimenting with tensor-network simulation backends to characterize where they fail for general oracles; and (4) running the algorithm on real superconducting hardware at small $n$ to validate the noiseless simulation against physical decoherence.

## References


[1] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.

[2] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997.

[3] M. S. Anis et al. (IBM Quantum). Qiskit: An open-source framework for quantum computing. https://qiskit.org, 2021.

[4] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik. qHiPSTER: The quantum high performance software testing environment. *arXiv:1601.07195*, 2016.

[5] T. Häner and D. S. Steiger. 0.5 petabyte simulation of a 45-qubit quantum circuit. *Proceedings of SC17: International Conference for High Performance Computing*, 2017.

[6] G. F. Viamontes, I. L. Markov, and J. P. Hayes. Improving gate-level simulation of quantum circuits. *Quantum Information and Computation*, 3(5):383–406, 2003.

[7] S. Isakov et al. (Google). Simulations of quantum circuits with approximate noise using qsim and Cirq. *arXiv:2111.02396*, 2021.

[8] NVIDIA Corporation. cuQuantum SDK: High-performance quantum circuit simulation. https://developer.nvidia.com/cuquantum-sdk, 2022.

[9] D. Gottesman. The Heisenberg representation of quantum computers. *Proceedings of the XXII International Colloquium on Group Theoretical Methods in Physics*, 1998.

[10] I. L. Markov and Y. Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.

[11] I. L. Chuang, N. Gershenfeld, and M. Kubinec. Experimental implementation of fast quantum searching. *Physical Review Letters*, 80(15):3408–3411, 1998.

[12] C. Figgatt et al. Complete 3-qubit Grover search on a programmable quantum computer. *Nature Communications*, 8:1918, 2017.